

 Add Comment

3/30/2026 to 4/21/2026

▼ Details

In this assignment, you will implement the evaluator for our language. This is the final step needed to have a "working" programming language which evaluates the parsed AST. The majority of this assignment is ensuring evaluation following the correct evaluation order, control flow, and scoping.

Setup/Submission

(<https://ufl.instructure.com/courses/555493/files/103969638/download>)

- Setup: [PlcProject \(Analyzer Patch\).zip](https://ufl.instructure.com/courses/555493/files/105760975?wrap=1) (<https://ufl.instructure.com/courses/555493/files/105760975?wrap=1>) ↓ (https://ufl.instructure.com/courses/555493/files/105760975/download?download_frd=1) contains a **patch** of new/updated source files. These files should be merged in with your *existing* project, *replacing* any existing files (the files being replaced are part of our provided API structure and should not be being modified). Copying the `src` folder into the root project should match the path structure properly.
- Submission: You will submit a zip of **only** the `src` folder, which includes both sources and tests. Using `Right Click src` + `Compress to Zip` should have the expected behavior, or alternatively `zip -r submission.zip src` from the command line. The name of the file is irrelevant.
 - The first test submission is **Sunday, April 12:**
 - Only "core" rules are graded (see spec).
 - The second test submission is **Thursday, April 16:**
 - All rules are graded. As a TEST 2 submission, this cannot be dropped!
 - The final submission is **Monday, April 20**. The final day to submit late is **Tuesday, April 21** (to allow discussion in lecture).

Analyzer Overview

Recall that the job of semantic analysis is determine if the semantic meaning of the AST is valid according the specification of the language. This can generally be broken down into the following categories:

- Structural AST requirements that *could* have been enforced by the Parser, but make more sense to handle within semantic analysis (typically best when the meaning is clear but the operation itself is invalid to allow IDE to error gracefully).
- Resolution of types, variables, and functions; especially in "larger" programs that may require linking multiple source files together to identify where variables/functions are located so they can be compiled with references to the right locations.
- Type checking, which utilizes static typing to traverse the program and identify unsafe/invalid uses of data throughout the program. This is most helpful in large projects and in languages which have a sophisticated type system, otherwise it's easy to end up "fighting" the compiler to get code that is correct dynamically to compile statically.
 - Our language will have just *slightly* above the bare minimum for types. In fact, there are many functions used in the Evaluator that we won't be able to represent accurate types for, e.g. `log` (which needs generic types to represent

that the argument and return value have the same type) and `list` (which needs functions that support taking a variable number of arguments, aka varargs, and thus can't be represented at all).

While our analyzer will be limited (especially the type system) the focus here is on how we can move validation from runtime in the evaluator to compile-time in the analyzer, even though we don't actually have runtime values. You should find that the analyzer is quite similar in logic/structure to the evaluator!

Crafting Interpreters

Unfortunately, crafting interpreters doesn't get into the aspects of semantic analysis or compile-time validation. This territory gets much closer into the "each language figures out it's own thing and they're all slightly broken" territory, *especially* with sophisticated type systems.

Grammar/Ast Changes

The grammar and AST of our language has been updated to support types. These types are optional in the AST (thus nothing should break in the Evaluator) however you will need to update your Parser accordingly. These changes will be included in Analyzer tests but only graded at the end of the semester in the Project Finale.

```
let_stmt ::= 'LET' identifier (':' identifier)? ('=' expr)? ';'
def_stmt ::= 'DEF' identifier '(' (identifier (':' identifier)? (',' identifier (':' identifier)?)*? ')' (':' identifier)? 'D
O' stmt* 'END'
```

Ast Types

The following table covers all AST types, highlighting behavior that is specific to semantic analysis

- Pay close attention to callouts relating to different behavior from the Evaluator, especially Limitations, to ensure you have an accurate understanding of the functionality our language is expected to support. If you have questions, please ask.
- Note that the `Dynamic` type effectively bypasses typechecking. This has the correct behavior in simple cases, but in more complex one (like trying to check variations on types in `Expr.Binary`) this can result in confusing behavior.

AST Type	Description	Examples
<code>Source</code> (provided)	Analyzes a full program, consistent with Evaluator validation except: • <code>RETURN</code> outside a function is now handled, see <code>Stmt.Return</code>. Returns the type of the last statement.	<pre>log(1); log(2); log(3);</pre>
<code>Stmt.Let</code> (core)	Analyzes a <code>LET</code> statement, consistent with Evaluator validation except: • The <code>type</code> of the variable is the first of: The ast's <code>type</code> (added to our grammar) if present, otherwise the value's <code>type</code> if present, else <code>Dynamic</code>. • The value's type if present must be a subtype of the variable type. • The context <code>uninitialized</code> value is updated accordingly to represent whether this field is initialized or not. • Returns the type of the variable when uninitialized, not <code>Nil</code>.	<pre>LET name; LET name = "value";</pre>

Stmt.Def	Analyzes a DEF statement, consistent with the Evaluator except: - All parameter types are the corresponding type in the ast's parameterTypes (added to our grammar) if present, else Dynamic. - The return type is the ast's returnType if present, else Dynamic. - When analyzing the function body, ensure: - The context function is set, for use with Return. - The function is defined in the current scope, allowing for recursion. <u>Limitation</u>: Functions defined later in the program will be considered undefined as we are not hoisting functions.	<pre>DEF name() DO log(1) END DEF name(param) DO log(param) END</pre>
Stmt.If (core)	Analyzes an IF statement, consistent with Evaluator validation except: Returns the type Dynamic, as the executed branch is unknown until evaluation. Note: IF requires the Context to be managed across two conditional branches. This is a key place to implement this logic that affects initialization and control flow analysis!	<pre>IF TRUE DO log("true"); END IF FALSE DO log(1) ELSE log(2); END</pre>
Stmt.For	Analyzes a FOR loop, consistent with Evaluator validation except: <u>Limitation</u>: The type of the FOR variable will <i>always</i> be Integer, as our type system does not support generics for Iterable.	<pre>FOR i IN range(1, 5) DO log(i); END</pre>
Stmt.Return	Analyzes a RETURN statement, consistent with Evaluator validation except: - The context function is set, otherwise the return is outside a function. - The value's type, else Nil, must be a subtype of the function's return type.	<pre>DEF name() DO RETURN "value"; END name();</pre>
Stmt.Expression (provided)	Analyzes an expression statement, consistent with Evaluator validation.	<pre>variable; //okay! function();</pre>
Stmt.Assignment (variables core)	Analyzes an assignment statement, consistent with Evaluator validation except: - For variables, the context uninitialized set is updated accordingly to represent this field as being initialized. - For properties, the property must be defined on the object following Prototypal Inheritance. A receiver / prototype of type Dynamic should permit any property.	<pre>variable = value; object.property = value;</pre>
Expr.Literal (provided)	Analyzes a literal expression, consistent with Evaluator validation.	<pre>1 "string"</pre>

Expr.Group (core)	Analyzes a group expression, consistent with Evaluator validation.	(expr)
Expr.Binary (core)	Analyzes a binary expression, consistent with Evaluator validation except: + / - / * / / : Consider the Dynamic type <i>carefully</i>, as isSubtypeOf will also catch Dynamic. The result type should only be Dynamic if both arguments are themselves Dynamic. == / != : One operand must be a subtype of the other, as otherwise the values should never be equal.	<pre>1 + 2 1 * 2 1 + 2 * 3 // 1 + (2 * 3) 1 * 2 + 3 // (1 * 2) + 3</pre>
Expr.Variable (core)	Analyzes a variable, consistent with Evaluator validation except: The variable must not be uninitialized in the current Context.	variable
Expr.Property	Analyzes a property, consistent with Evaluator validation except: The property must be defined on the object following Prototypal Inheritance. A receiver / prototype of type Dynamic should permit any property.	object.property
Expr.Function (core)	Analyzes a function, consistent with Evaluator validation except: The argument arity/types must be permitted by the function. Note: Functions cannot be declared without a definition in our language, hence there is no need to check if the function is uninitialized.	<pre>function() function(1, 2, 3)</pre>
Expr.Method	Analyzes a method, consistent with Evaluator validation except: - The property must be defined on the object following Prototypal Inheritance. A receiver / prototype of type Dynamic should permit any property. - The argument arity/types must be permitted by the method. IMPORTANT: In our Analyzer, the Type.Function for methods will <i>not</i> contain a receiver parameter, unlike the Evaluator. It's not necessary and can add complications related to recursive structures.	<pre>object.method() object.method(1, 2, 3)</pre>
Expr.ObjectExpr	Analyzes an object, with the following behavior: - Fields will adapt the same analysis behavior as LET, with the type being defined to the ObjectType. <u>Limitation</u>: Uninitialized fields will not be checked, as objects do not have constructors like classes that would enforce initialization. - Methods will adapt the same analysis behavior as DEF, with the type being defined to the ObjectType. IMPORTANT: In our Analyzer, the Type.Function for methods will <i>not</i> contain a receiver parameter, unlike the	<pre>OBJECT DO LET field; END OBJECT DO DEF method() DO END END</pre>

	Evaluator. It's not necessary and can add complications related to recursive structures. Tip: It is not recommended to try reusing <code>LET</code> / <code>DEF</code> for this - just copy the relevant behavior and adjust accordingly using tests to verify semantics.	
--	---	--

Miscellaneous "Hooks"

The bottom of the Analyzer contains fields/functions that "hook" into parts of the provided classes (similar to `sqrt` / `range` in the Evaluator). Most of these contribute to the behavior of Semantic Analysis as defined above, so cross-reference concepts accordingly.

<code>mergeUninitialized</code> (core)	Merges the <code>uninitialized</code> context value for branching children of a parent Context. The returned value should represent the variables that are uninitialized when considering all possible branches of execution. Hint: Consider each scenario - what is the expected value? Is there a pattern?	<pre>LET x; IF TRUE DO x = 1; END LET x; IF TRUE DO x = 1; ELSE x = 2; END</pre>
<code>mergeReturns</code>	Merges the <code>returns</code> context value for branching children of a parent Context. The returned value should represent whether control flow returns or not when considering all possible branches. Hint: Consider each scenario - what is the expected value? Is there a pattern?	<pre>IF TRUE DO RETURN; END IF TRUE DO RETURN; ELSE RETURN; EN D</pre>
<code>SQRT</code>	The type of <code>sqrt</code> , which takes and returns a <code>Decimal</code> .	<code>sqrt(4.0)</code>
<code>RANGE</code>	The type of <code>range</code> , which takes two <code>Integer</code> s and returns an <code>Iterable</code> .	<code>range(1, 10)</code>
<code>isSubtypeOf(subtype, supertype)</code> (non-prototype core)	Returns <code>true</code> if this a valid subtype relationship, as defined by: • All types are subtypes of themselves (per <code>.equals()</code>). • All types are subtypes of <code>Any</code> (similar to Java's <code>Object</code>). • All types are subtypes <u>and supertypes</u> of <code>Dynamic</code> (gradual typing). • <code>Nil</code>, <code>Comparable</code>, <code>Iterable</code>, and any <code>ObjectType</code> are subtypes of <code>Equatable</code>. • <code>Boolean</code>, <code>Integer</code>, <code>Decimal</code>, <code>Character</code>, and <code>String</code> are subtypes of <code>Comparable</code>. • An <code>ObjectType</code> is a subtype of another <code>ObjectType</code> if it's <code>prototype</code> field is a subtype, following Prototypal Inheritance. More formally, <code>isSubtypeOf(object.prototype, supertype)</code> implies <code>isSubtypeOf(object, supertype)</code>.	<pre>isSubtypeOf(Integer, Integer): tru e isSubtypeOf(Integer, Decimal): fal se isSubtypeOf(Integer, Any): true isSubtypeOf(Dynamic, Integer): tru e isSubtypeOf(Integer, Equatable): t rue isSubtypeOf(object, object.prototy pe): true</pre>

Changelog

Date	Changes
------	---------

3/31	- Updated provided code (PlcProject (Analyzer Patch).zip (https://ufl.instructure.com/courses/555493/files/105760975?wrap=1)  (https://ufl.instructure.com/courses/555493/files/105760975/download?download_frd=1)) with tests & other fixes/improvements: <code>Analyzer</code>: Updated <code>Ast.Source</code> to include proper return value, removed misleading TODO. <code>Environment</code>: Fixed duplicate declaration for <code>sqr</code> / <code>range</code> crashing <code>Main.java</code>. <code>AnalyzerTests</code>: Added. Note the separation of <code>testContext</code> behavior for "core" Ast tests. <code>ParserTypeTests</code>: Added. - Documented inconsistencies with Evaluator return behavior for Let (when uninitialized) and If (conditional evaluation). - Marked <code>isSubtypeOf</code> (excl. prototypal inheritance) and <code>mergeUninitialized</code> as core for Test 1.
4/2	Fixed example for <code>isSubtypeOf</code> indicating <code>Strig</code> is a subtype of <code>Integer</code>.



<https://ufl.instructure.com/courses/555493/modules/items/12806053>



<https://ufl.instructure.com/courses/555493/modules/>